

PCS956-TS-companion_A

September 18, 2026

0.1 PCS956 time series companion A: inspection and basic EDA

This notebook is ‘companion A’ for the PCS956 time series lectures. It focuses on how to go from a time-stamped data file to a first inspection of:

- the structure of the time index and sampling pattern
- aspects of data quality that are visible from the data alone
- simple plots of levels and differences
- basic autocorrelation diagnostics
- simple decompositions and persistence baselines

The emphasis here is on well-behaved example series that let us see how inspection and basic modelling workflows look in a straightforward setting. In real applications we may encounter much messier data. The intention is that you start from the templates in this notebook and adapt them carefully to your own setting, using domain knowledge where needed.

Spectral (frequency-domain) methods and more advanced models will be covered in later companions.

0.2 Data sources and formats

The datasets in this companion are derived from the R package `astsa`:

Shumway, R.H. and Stoffer, D.S. `astsa`: Applied Statistical Time Series Analysis

GitHub: <https://github.com/nickpoison/astsa>

CRAN: <https://cran.r-project.org/package=astsa>

They have been exported to CSV and placed in a local `data/` folder:

- `../data/soi.csv` - Southern Oscillation Index (monthly, univariate)
- `../data/ENSO.csv` - ENSO index (monthly, can be treated as multivariate)
- `../data/gtemp_both.csv` - global temperature anomalies (annual, univariate)
- `../data/djia.csv` - Dow Jones Industrial Average (daily, multivariate)
- `../data/eqexp.csv` - seismic traces for earthquake vs explosion (classification-type)

Each example section below is self-contained: it includes its own imports, reads one CSV file, and performs basic inspection and EDA. You can copy any section into your own notebook and adapt the path and column names.

These `astsa` datasets illustrate how workflows look when the time index behaves in a simple way and the series are well-suited for basic time-series inspection and modelling.

In the later constructed examples (Sections 7–8), we use simulated series with:

- **missing values** (isolated points and contiguous blocks), and
- **level shifts vs isolated spikes**,

to make it easier to recognise these features when they occur in real data.

The goal in this companion is not to fit sophisticated models, but to:

- develop a systematic habit of inspecting the time index, data quality, and basic temporal structure, and
- use simple tools (plots, rolling summaries, ACF/PACF, decompositions, and persistence baselines) to distinguish between long-run trends, seasonal patterns, level shifts, and more noise-like behaviour.

Later companions will build on this workflow with more advanced models and spectral (frequency-domain) methods.

0.2.1 Overview of the inspection workflow

Each template in this companion follows the same basic inspection workflow. Starting from a time-stamped data file (typically CSV), we move through:

1. **Loading and basic structure**
 - Read the data from file.
 - Ensure that the time index is correctly represented (for example, as a `DatetimeIndex` for calendar data, or as an integer index for sample sequences).
 - Check the ordering and inferred sampling frequency.
2. **Data-quality checks**
 - Count missing values and check for duplicated time stamps.
 - Inspect simple summary statistics (minimum, maximum, mean, quartiles).
 - Identify obvious glitches or physically implausible values.
3. **Visual inspection of levels and differences**
 - Plot the series in levels to see overall behaviour.
 - Plot first differences (or returns) to see how changes behave.
 - Use rolling mean and rolling standard deviation to highlight changes in typical level or volatility over time.
4. **Autocorrelation diagnostics**
 - Plot autocorrelation (ACF) and partial autocorrelation (PACF) of levels.
 - Plot ACF/PACF of differences or returns.
 - Compare how much temporal dependence remains after differencing.
5. **Simple decompositions and baselines (where appropriate)**
 - For regular seasonal data (such as monthly SOI), perform a simple additive decomposition into trend, seasonal, and residual components.
 - Inspect residuals and their ACF to see what structure remains after removing trend and seasonal patterns.
 - For forecasting-style problems, evaluate a very simple persistence baseline (one-step-ahead forecast equal to the last observed value) and inspect the baseline residuals and

their autocorrelation.

6. Problem-specific views

- For multivariate series (such as ENSO), look at correlations between components and focus EDA on one chosen variable.
- For classification-style data (such as earthquake vs explosion traces), compare representative traces from different groups, and consider how time-series signals might be turned into features.

In the later constructed examples (Sections 7–8), we use simulated series with:

- **missing values** (isolated points and contiguous blocks), and
- **level shifts vs isolated spikes**,

to make it easier to recognise these features when they occur in real data.

The goal in this companion is not to fit sophisticated models, but to:

- develop a systematic habit of inspecting the time index, data quality, and basic temporal structure, and
- use simple tools (plots, rolling summaries, ACF/PACF, decompositions, and persistence baselines) to distinguish between long-run trends, seasonal patterns, level shifts, and more noise-like behaviour.

Later companions will build on this workflow with more advanced models and spectral (frequency-domain) methods.

0.2.2 How this companion relates to the lectures and other companions

This notebook corresponds to **Lecture TS1** (intro and overview) and focuses on:

- temporal indexing and sampling structure,
- data quality, missing values, anomalies, and level shifts,
- visual EDA for levels, differences, trend, seasonality, and autocorrelation,
- simple decompositions and a persistence baseline,
- basic intuition for stationarity and random-walk-like behaviour.

Later companions build on this foundation:

- **PCS956-TS-companion_B** (Lecture TS2): classical models and forecasting baselines (mean, persistence, seasonal naive, simple AR), AR/MA/ARIMA-type models used as baselines with residual checks, time series as supervised learning with lagged features (for example Random Forest on lags), temporal train/validation/test splits, evaluation on levels vs differences, and simple spectral views via periodograms.
- **PCS956-TS-companion_C** (Lecture TS3): anomaly detection, concept drift, multivariate dependence, and more advanced time-aware ML methods, building on the inspection and baseline workflow from Companions A and B.

When working on the mini-project, start from the inspection and EDA patterns in Companion A, then move to baselines and models in Companion B and ML methods in Companion C as needed.

0.3 1. Template: univariate monthly series (example: SOI)

This template shows how to:

- load a univariate series from CSV (`soi.csv`)
- inspect structure and basic data quality
- plot levels and differences
- plot ACF/PACF
- perform a simple decomposition to separate trend and periodic structure
- evaluate a persistence baseline

We will later extend this SOI template with basic residual diagnostics after decomposition and baselines.

SOI raw head:

	date	year	month	soi
0	1950-01-01	1950	1	0.377
1	1950-02-01	1950	2	0.246
2	1950-03-01	1950	3	0.311
3	1950-04-01	1950	4	0.104
4	1950-05-01	1950	5	-0.016

SOI info:

```
<class 'pandas.DataFrame'>
DatetimeIndex: 453 entries, 1950-01-01 to 1987-09-01
Data columns (total 1 columns):
#   Column  Non-Null Count  Dtype
---  -
0    soi      453 non-null    float64
dtypes: float64(1)
memory usage: 7.1 KB
None
```

SOI length: 453

SOI start: 1950-01-01 00:00:00

SOI end: 1987-09-01 00:00:00

Inferred frequency (SOI): MS

SOI first 3:

date	soi
1950-01-01	0.377
1950-02-01	0.246
1950-03-01	0.311

Name: soi, dtype: float64

SOI last 3:

date	soi
1987-07-01	-0.388
1987-08-01	0.388
1987-09-01	0.071

Name: soi, dtype: float64

SOI missing values: 0

SOI summary statistics:

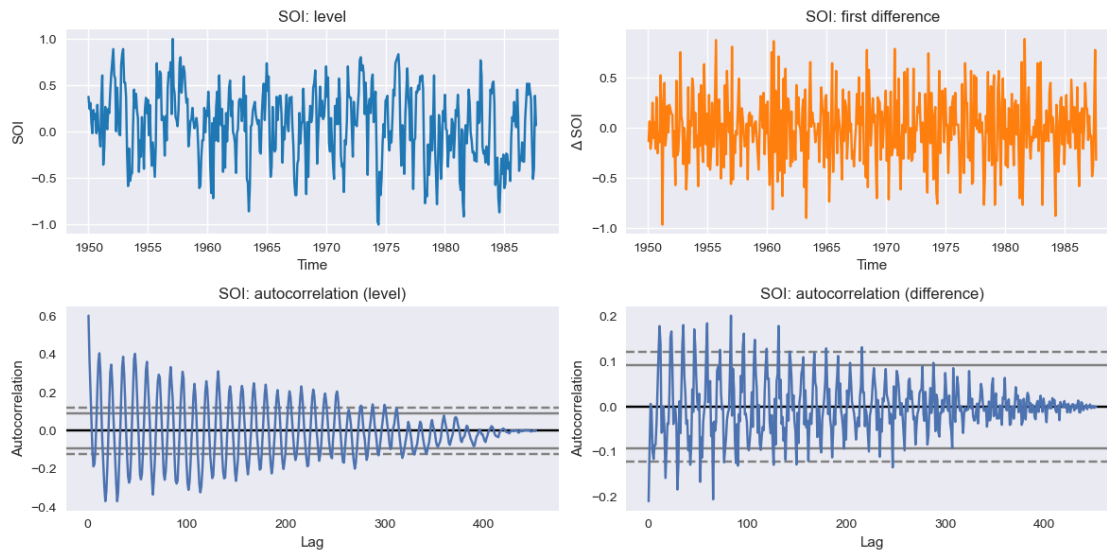
	count	mean	std	min	25%	50%	75%	max
soi	453.0	0.080038	0.382746	-1.0	-0.18	0.115	0.366	1.0

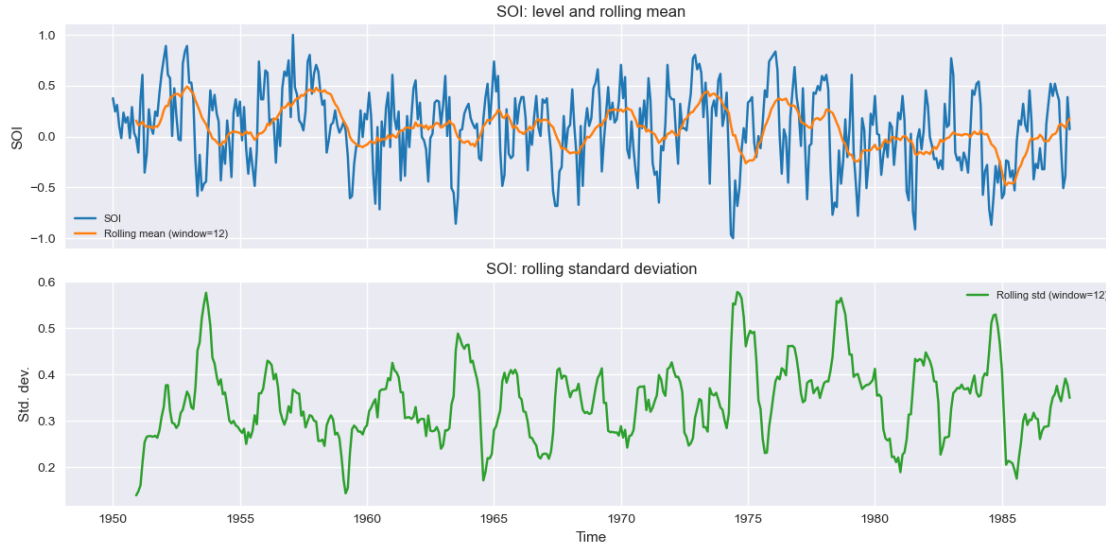
SOI duplicated index values: 0

SOI range:

min = -1.0

max = 1.0





0.3.1 Short note: rolling windows for mean and standard deviation

In several examples we use **rolling windows** to compute running summaries such as:

- rolling mean (average over a recent window),
- rolling standard deviation (a simple measure of local variability).

For a window of size w (for example $w = 12$ months), the rolling mean at time t is:

$$\bar{X}_t^{(w)} = \frac{1}{w} \sum_{j=0}^{w-1} X_{t-j},$$

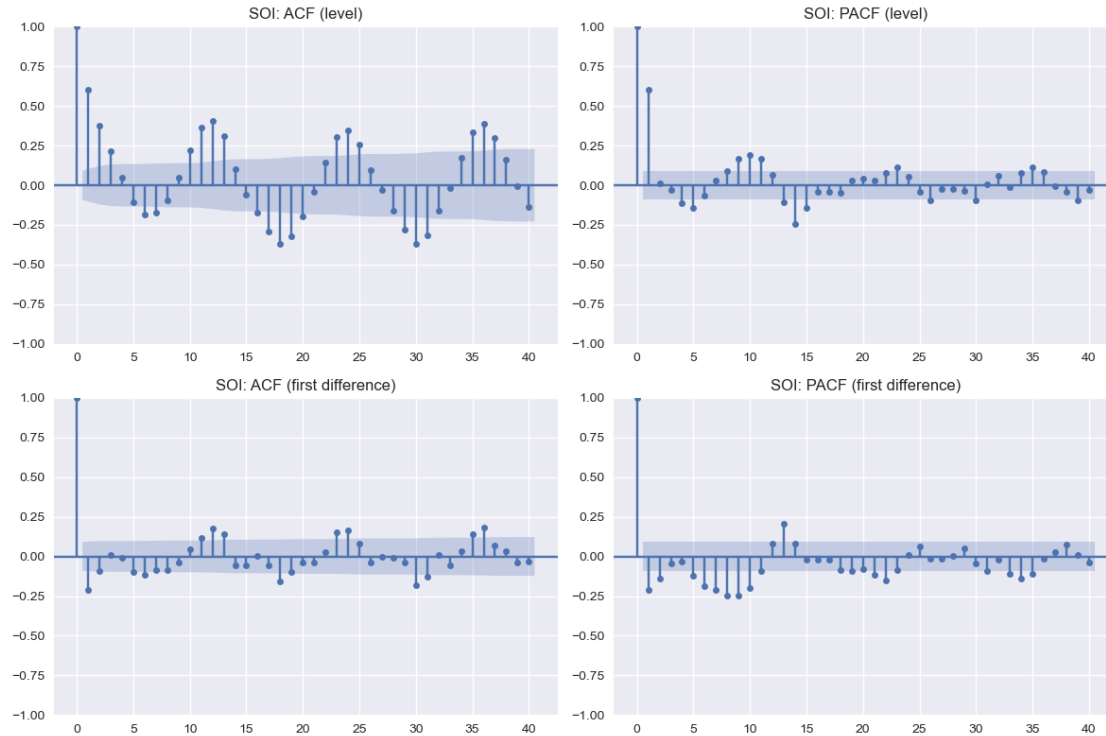
and the rolling standard deviation is the usual sample standard deviation computed over the same window. These summaries are helpful for:

- visualising **slow changes in typical level** (trend or regime changes),
- seeing **changes in variability** (for example calm vs turbulent periods).

Rolling windows have some important limitations:

- the choice of window size w is **arbitrary and context-dependent**:
 - too small: very noisy behaviour;
 - too large: genuine changes can be smoothed away.
- they are primarily **exploratory tools**, not formal tests; they give hints about changes, but do not prove that a process has become stationary or non-stationary.
- near the start of the series, rolling statistics are based on fewer points, so edge behaviour should be interpreted with caution.

In this companion rolling means and standard deviations are used mainly as **visual diagnostics** to highlight possible changes in level or volatility, not as modelling assumptions.



0.3.2 Autocorrelation with anomalies and missing values

In these curated examples the series themselves are **clean**: there are no genuine missing values in the original data files, and no obvious anomalies. Any NaN values we encounter come from simple transformations such as:

- first differences (`series.diff()`), which introduce a NaN at the start,
- decompositions, which can produce NaN at the ends of trend or residual components.

Before computing autocorrelations or partial autocorrelations we therefore use:

- `series.dropna()` to remove those transformation-induced NaN values, and
- pass only the finite-valued series to functions such as `plot_acf`, `plot_pacf`, and `autocorrelation_plot`.

In real applications we may face two additional issues:

- **Anomalies (spikes, glitches):**
 - A few large spikes can distort sample autocorrelations, especially at short lags.
 - Before treating ACF/PACF as describing “typical behaviour”, we should inspect the series and decide whether obvious anomalies should be removed, adjusted, or explicitly discussed as part of the story (for example, interventions or rare events).
- **Genuine missing values (NaN) in the raw data:**
 - Standard time-series tools (autocorrelations, spectral estimates, many model fits) do not work with NaN values; they require finite observations.

- Simply dropping all missing values (`dropna()`) can be acceptable when there are **very few isolated gaps in a long series**, but it may change the effective sampling pattern and reduce reliability when gaps are more substantial.

In this companion we:

- compute ACF/PACF only on finite-valued series (after removing the transformation-induced NaNs),
- treat serious missingness and anomalies as **features to be described** (see Sections 7–8),

rather than silently ignoring them. In your own mini-project, you should explicitly describe how missing values and anomalies are handled before computing autocorrelations or spectra, and avoid using “ignore NA” defaults without thinking about their effect on the analysis.

ADF test (SOI level):

```
Test statistic: -5.364
p-value:       0.000
Used lags:     14
Number of obs.: 438
Critical values:
  1%: -3.445
  5%: -2.868
 10%: -2.570
```

Interpretation: reject unit-root null at 5% level (evidence for stationarity).

ADF test (SOI first difference):

```
Test statistic: -6.231
p-value:       0.000
Used lags:     13
Number of obs.: 438
Critical values:
  1%: -3.445
  5%: -2.868
 10%: -2.570
```

Interpretation: reject unit-root null at 5% level (evidence for stationarity).

0.3.3 Optional note: formal stationarity tests

The Augmented Dickey–Fuller (ADF) test is one way to check for a unit root in a time series. In its simplest form, it tests:

H_0 : series has a unit root (is non-stationary) H_1 : series is stationary (no unit root).

In this notebook, we use the ADF test as a **supporting tool**:

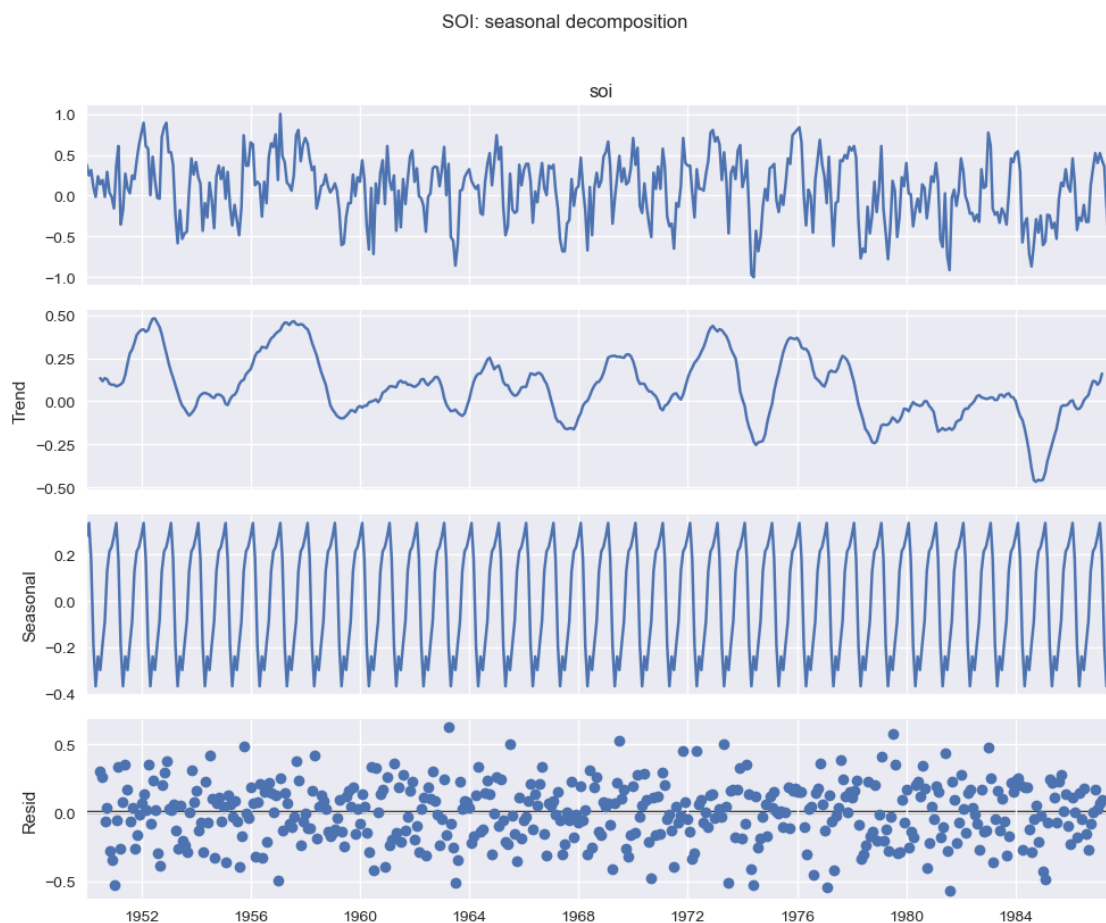
- we already look at levels, differences, and ACF/PACF plots,
- the ADF test can be used to see whether these visual impressions are consistent with a formal unit-root test.

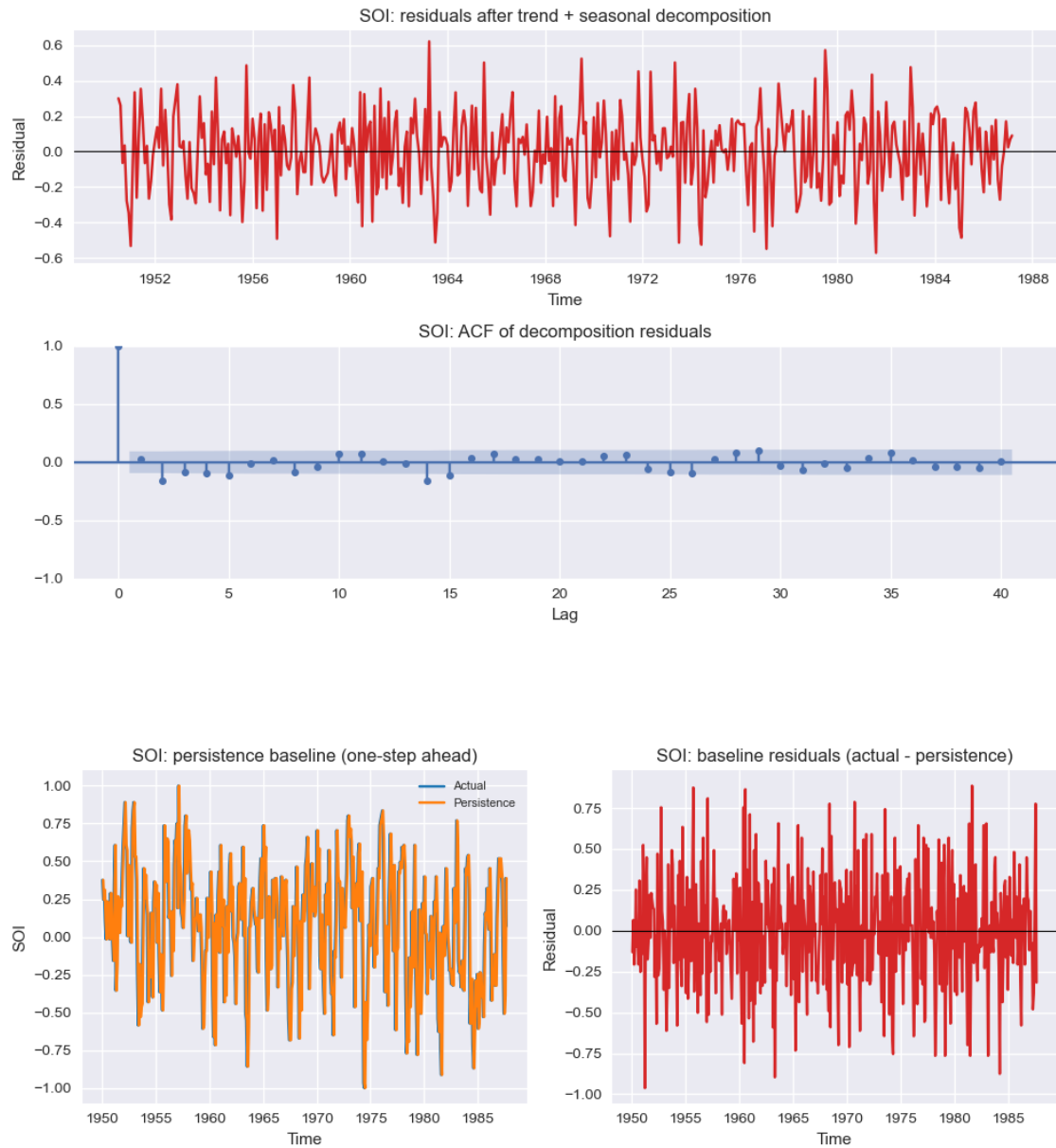
It is important to remember that:

- the ADF test has assumptions (for example, about the form of the trend and the error structure),
- failing to reject H_0 does not prove that a series is “non-stationary” in every sense,
- passing the test does not guarantee that the series behaves like a simple, stationary model.

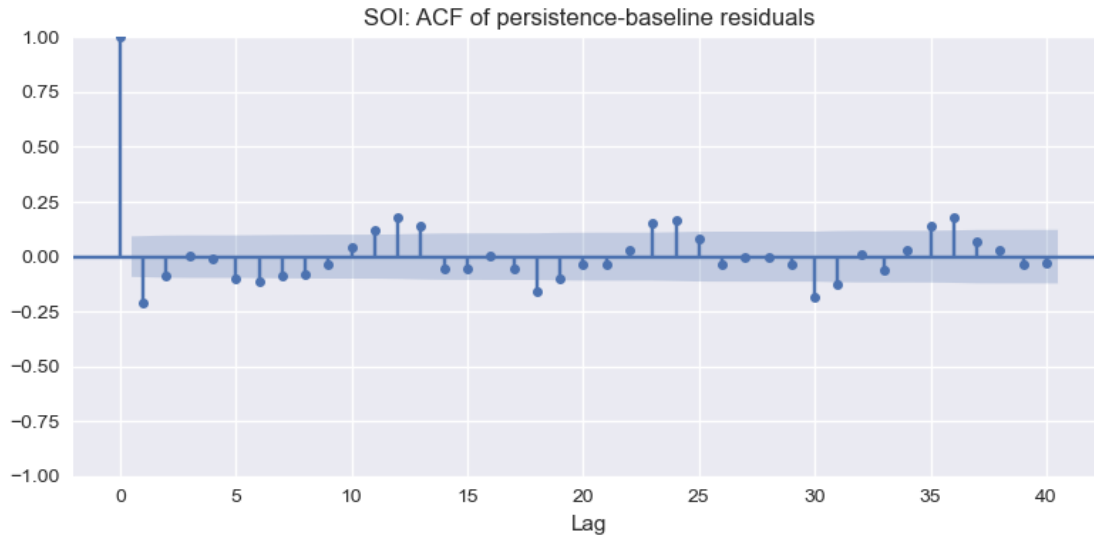
In practice, tests such as ADF are used alongside visual diagnostics and domain knowledge, not as a substitute for them.

In Lecture TS1 (Section 6.1) we discuss tests such as ADF as heuristics for unit-root behaviour. Here you can see how they line up with the visual diagnostics in this companion, treating them as guides rather than oracles.





SOI persistence baseline:
MAE: 0.26953539955752215
RMSE: 0.3402925818867763



0.4 2. Template: multivariate monthly series (example: ENSO)

This template shows:

- loading `ENSO.csv` as a multivariate time series
- basic inspection and correlation
- EDA for one chosen variable

Later extensions could include:

- a quick “spaghetti” plot of all numeric ENSO variables
- a basic scatter plot to illustrate dependence between two components

ENSO raw head:

	date	year	month	ENSO
0	1951-01-01	1951	1	2.007800
1	1951-02-01	1951	2	1.123289
2	1951-03-01	1951	3	-0.317529
3	1951-04-01	1951	4	-0.805767
4	1951-05-01	1951	5	-1.142218

ENSO info:

```
<class 'pandas.DataFrame'>
```

DatetimeIndex: 862 entries, 1951-01-01 to 2022-10-01

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	year	862 non-null	int64
1	month	862 non-null	int64
2	ENSO	862 non-null	float64

```
dtypes: float64(1), int64(2)
memory usage: 26.9 KB
None
```

ENSO correlation matrix (numeric columns):

	year	month	ENSO
year	1.000000	-0.005774	0.053934
month	-0.005774	1.000000	-0.034420
ENSO	0.053934	-0.034420	1.000000

0.4.1 Multivariate structure and cross-correlations (conceptual note)

For multivariate series, we care about **both**:

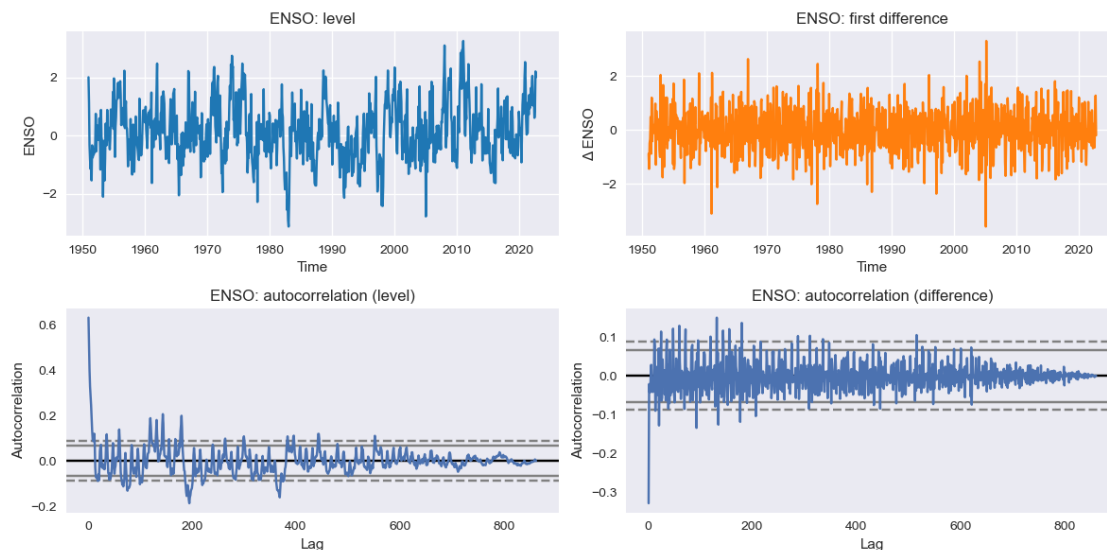
- how each component behaves over time (its own trend, seasonality, autocorrelation), and
- how components relate to each other (for example, whether changes in one tend to precede or follow changes in another).

The simple correlation matrix above shows **static correlations** between components. In later materials (TS2/TS3 and their companions) we will also look at:

- **cross-correlation functions**, which describe how one series is related to lagged versions of another,
- and Granger-style predictive “causality” tools (still about prediction, not strong causal claims).

In this companion, the multivariate example is kept light:

- use the correlation matrix to see which variables move together on average,
- and then focus EDA on one chosen variable (ENSO or a similar index).



0.4.2 Plotting multivariate series: when simple plots are enough (and when they are not)

In this small ENSO example, plotting **one main variable** over time (with its differences and autocorrelation) is usually enough to illustrate basic time-series behaviour.

For multivariate series more generally, there are several common plotting strategies:

- **Spaghetti plots:** overlay several components in one plot.
 - Helpful when there are only a few series with compatible units and similar scales.
 - Can become hard to read if there are many dimensions, very long series, or very different magnitudes.
- **Small multiples:** separate plots for each component (or for a few selected components), arranged side by side or in a grid.
 - Clearer when units or scales differ, or when we want to focus on subsets of variables.
 - Still manageable only for modest numbers of series.
- **Interactive tools or dimension reduction:**
 - When there are many variables, static plots quickly become cluttered.
 - In such cases, interactive plotting, brushing/filtering tools, or dimension-reduction methods (for example PCA on features, embeddings) may be more appropriate for exploring structure.

In this companion multivariate plotting is deliberately kept simple, focusing on one main variable plus a correlation matrix. For high-dimensional series, we are expected to think about:

- which variables are most relevant to our question,
- whether they can reasonably share a plot and axis,
- and whether we need more advanced tools to explore multivariate dependence.

0.5 3. Template: annual series (example: global temperature)

This template focuses on an annual univariate series with a clear long-term trend.

Here we:

- overlay a simple linear trend fit to highlight non-stationarity in levels,
- add residual checks for a trend-only model (including an ACF of residuals).

`gtemp_both` raw head:

	date	year	gtemp_both
0	1850-01-01	1850	-0.24
1	1851-01-01	1851	-0.25
2	1852-01-01	1852	-0.27
3	1853-01-01	1853	-0.15
4	1854-01-01	1854	-0.05

`gtemp_both` info:

```
<class 'pandas.DataFrame'>
```

```
DatetimeIndex: 174 entries, 1850-01-01 to 2023-01-01
```

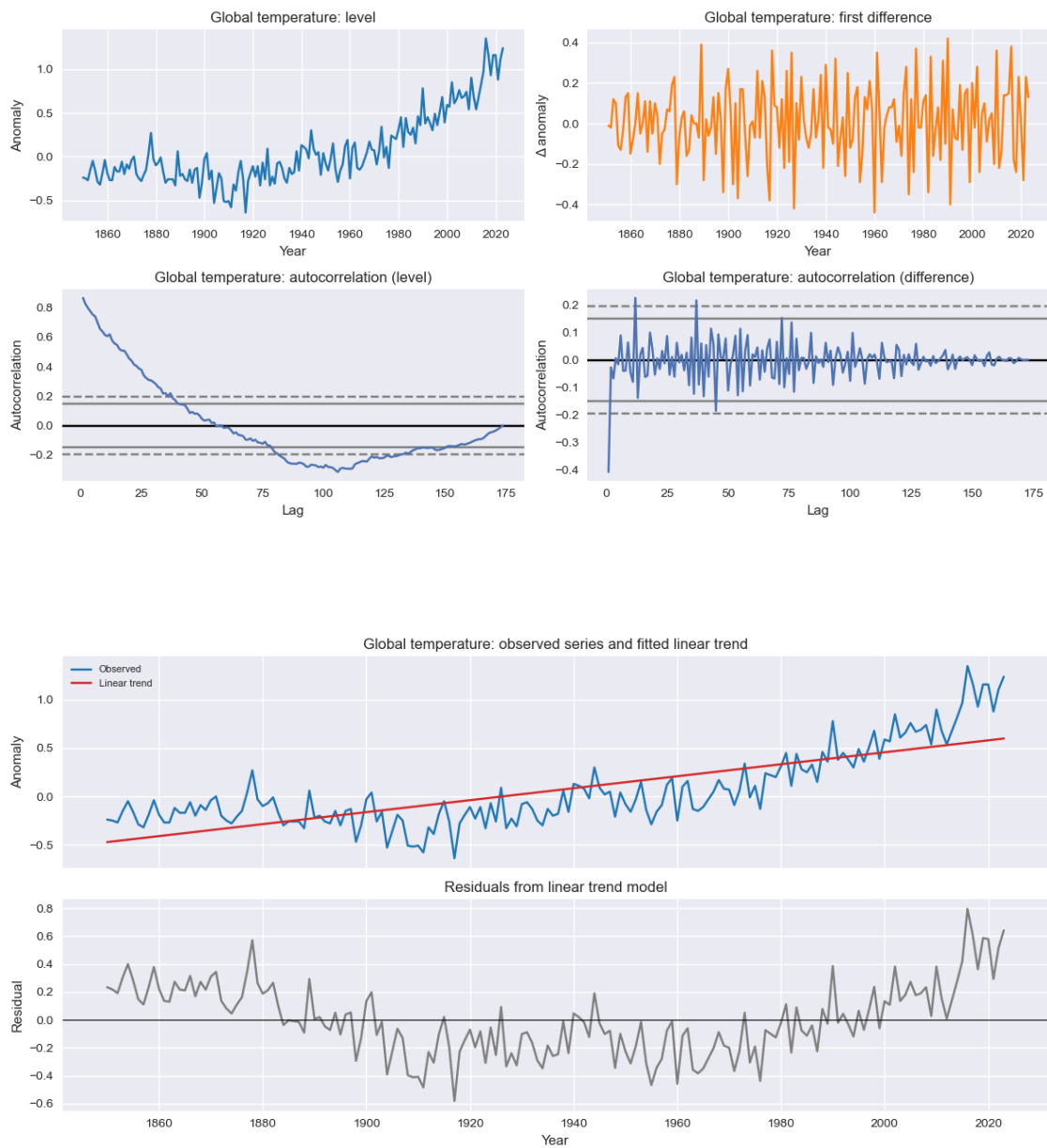
```
Data columns (total 1 columns):
```

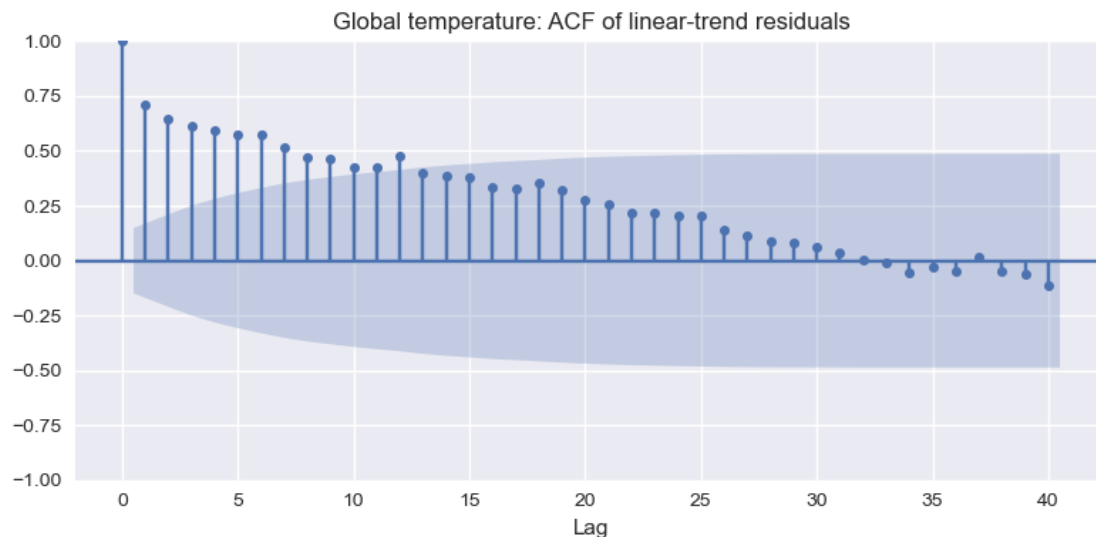
#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

```

-----
0    gtemp_both  174 non-null    float64
dtypes: float64(1)
memory usage: 2.7 KB
None

```





0.6 4. Template: daily financial series (example: DJIA Close)

This template uses a daily financial series, where levels often look like a random walk and differences or returns are closer to noise.

We see:

- levels and first differences,
- ACF on both,
- log-returns and their autocorrelation (often near zero at most lags, in line with the random-walk discussion in the lecture),
- a rolling volatility estimate to visualise calm vs turbulent periods.

DJIA raw head:

	date	Open	High	Low	Close \
0	1970-01-02	11278.530273	11384.110352	11275.049805	11342.889648
1	1970-01-03	11343.450195	11405.879883	11316.790039	11347.450195
2	1970-01-04	11346.809570	11359.700195	11305.830078	11336.320312
3	1970-01-05	11336.559570	11355.370117	11260.839844	11283.250000
4	1970-01-06	11283.250000	11379.870117	11282.769531	11354.490234

	Volume
0	336420000.0
1	325090000.0
2	232000000.0
3	289230000.0
4	270270000.0

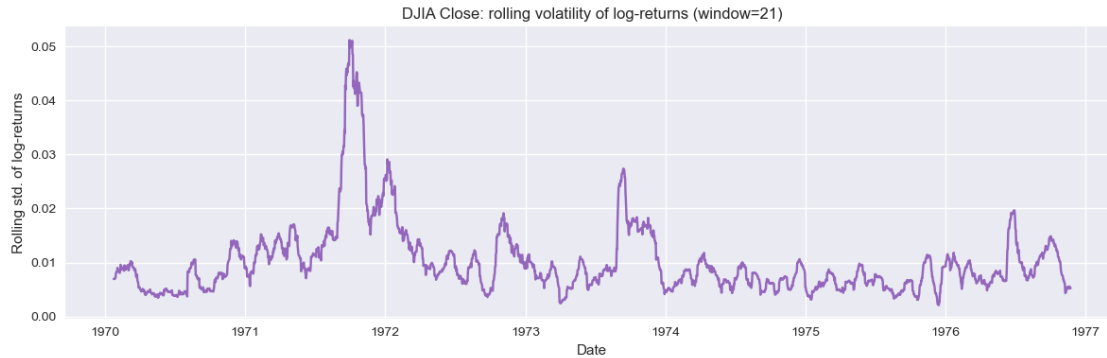
DJIA Close info:

```

<class 'pandas.DataFrame'>
DatetimeIndex: 2518 entries, 1970-01-02 to 1976-11-23
Data columns (total 1 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   Close    2518 non-null       float64
dtypes: float64(1)
memory usage: 39.3 KB
None

```





0.6.1 Rolling volatility as a visual diagnostic

Here we use a rolling standard deviation of log-returns (over about one trading month) to highlight periods of higher or lower variability. This follows the same idea as the rolling mean/std in Section 1.5: a **moving summary** over a chosen window, used for visual diagnostics rather than formal testing.

0.6.2 Connection to TS1: random-walk-like behaviour

The DJIA Close series illustrates the TS1 discussion of random walks: - levels look like a smooth, trending series with apparent structure, - differences and log-returns are much closer to noise, - ACF of returns is often near zero at most lags.

This is why persistence baselines and modelling on differences/returns are central in financial time-series work.

0.7 5. Template: classification-style data (example: eqexp)

This template:

- loads `eqexp.csv` with sample index `t`
- selects one earthquake channel (for example EQ1) and one explosion channel (EX1)
- plots them for visual comparison

No classification model is fitted here; this is purely EDA.

Later we may add:

- overlay of multiple earthquake traces and multiple explosion traces
- simple features (for example, energy, maximum amplitude) for two example traces, to illustrate how time-series signals can be turned into features for classification

`eqexp raw head:`

	t	EQ1	EQ2	EQ3	EQ4	EQ5	EQ6	EQ7	\
0	1	0.000818	-0.208034	-0.020997	-0.064262	0.134687	0.050881	-0.019262	
1	2	0.033801	0.030319	-0.041993	-0.016235	0.086986	0.076963	-0.020047	
2	3	0.016628	0.099049	-0.068165	-0.085908	0.116168	0.082174	-0.011505	

```

3 4 0.004634 0.203353 -0.067869 -0.098760 0.113362 0.058811 -0.025916
4 5 -0.036527 0.540785 -0.034452 -0.040586 0.048824 0.013746 -0.026069

```

```

      EQ8      EX1      EX2      EX3      EX4      EX5      EX6  \
0 -0.021916 0.041018 0.042617 -0.008053 -0.073002 -0.110650 -0.027236
1 -0.252665 0.030043 0.026397 -0.001922 -0.094665 -0.098188 -0.009590
2 -0.171574 0.023733 0.025761 -0.011164 -0.114399 -0.078099 -0.033374
3 -0.144022 0.012895 0.033712 -0.018302 -0.090807 -0.064241 -0.006138
4 -0.319667 0.002607 0.014948 -0.013086 -0.031011 -0.048235 -0.011892

```

```

      EX7      EX8      NZ
0 -0.056327 0.200717 0.362830
1 -0.085072 0.241603 0.338156
2 -0.111311 0.307270 0.014653
3 -0.103878 0.323377 -0.369302
4 -0.082565 0.209389 -0.383436

```

eqexp info:

```
<class 'pandas.DataFrame'>
```

RangeIndex: 2048 entries, 1 to 2048

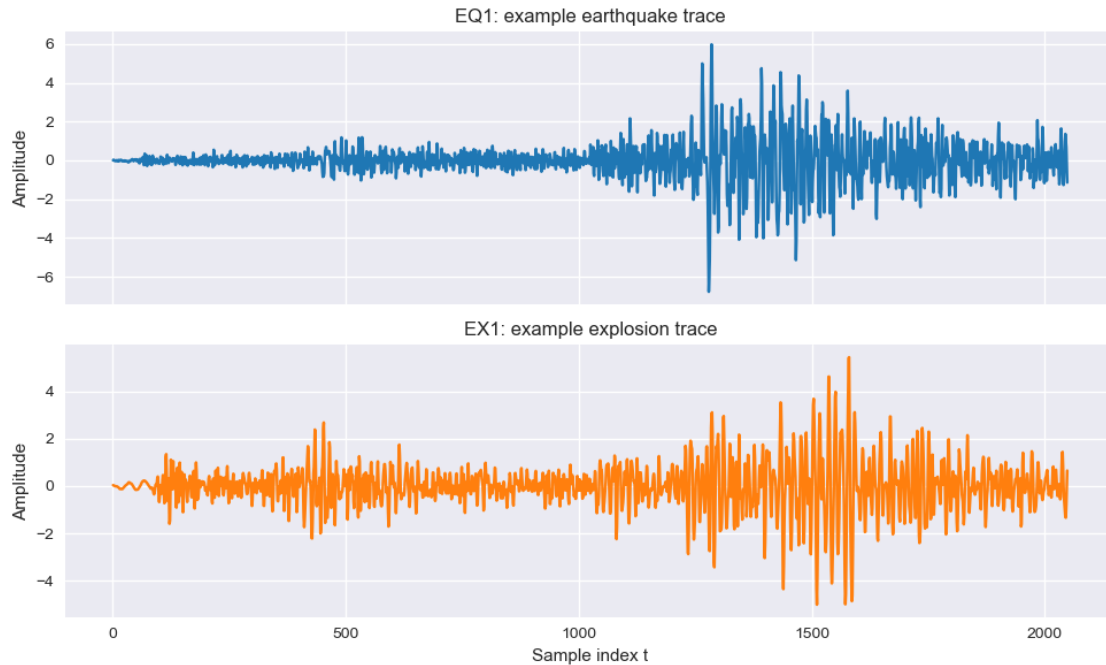
Data columns (total 17 columns):

#	Column	Non-Null Count	Dtype
0	EQ1	2048 non-null	float64
1	EQ2	2048 non-null	float64
2	EQ3	2048 non-null	float64
3	EQ4	2048 non-null	float64
4	EQ5	2048 non-null	float64
5	EQ6	2048 non-null	float64
6	EQ7	2048 non-null	float64
7	EQ8	2048 non-null	float64
8	EX1	2048 non-null	float64
9	EX2	2048 non-null	float64
10	EX3	2048 non-null	float64
11	EX4	2048 non-null	float64
12	EX5	2048 non-null	float64
13	EX6	2048 non-null	float64
14	EX7	2048 non-null	float64
15	EX8	2048 non-null	float64
16	NZ	2048 non-null	float64

dtypes: float64(17)

memory usage: 272.1 KB

None



0.8 7. Constructed example with missing values

In many real data sets, some values are missing. Once you see that the count of missing values is positive, it is useful to look more closely at when and how these gaps occur. One way to do this is to plot the series and highlight intervals where values are not observed (for example, using semi-transparent bands), so that patterns in the missing values become visible.

In this section we construct a simple example with:

- isolated missing values,
- and one or two contiguous blocks of missing values,

and illustrate:

- how these gaps appear in a basic time series plot when intervals with missing data are visually marked,
- how to recognise whether missingness tends to occur as isolated points, contiguous blocks, or seems to follow some other pattern.

Constructed missingness example:

Total length: 240

Missing values: 14

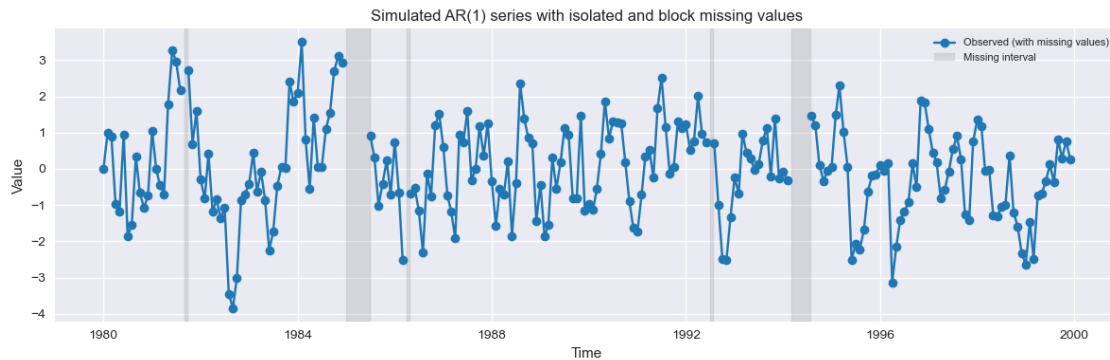
Missing blocks detected:

1981-09-01 to 1981-10-01

1985-01-01 to 1985-07-01

1986-04-01 to 1986-05-01

1992-07-01 to 1992-08-01
1994-03-01 to 1994-08-01



0.9 8. Simulated example with a level shift

Changes in the typical level of a series over time (for example, before and after a policy change or intervention) are easier to see first in a simple, controlled example. A simulated series with a known level shift can clarify how such changes appear in basic plots and rolling summaries.

In this section we construct a simple simulated series with:

- a constant mean in an initial period,
- a different mean in a later period (a level shift),

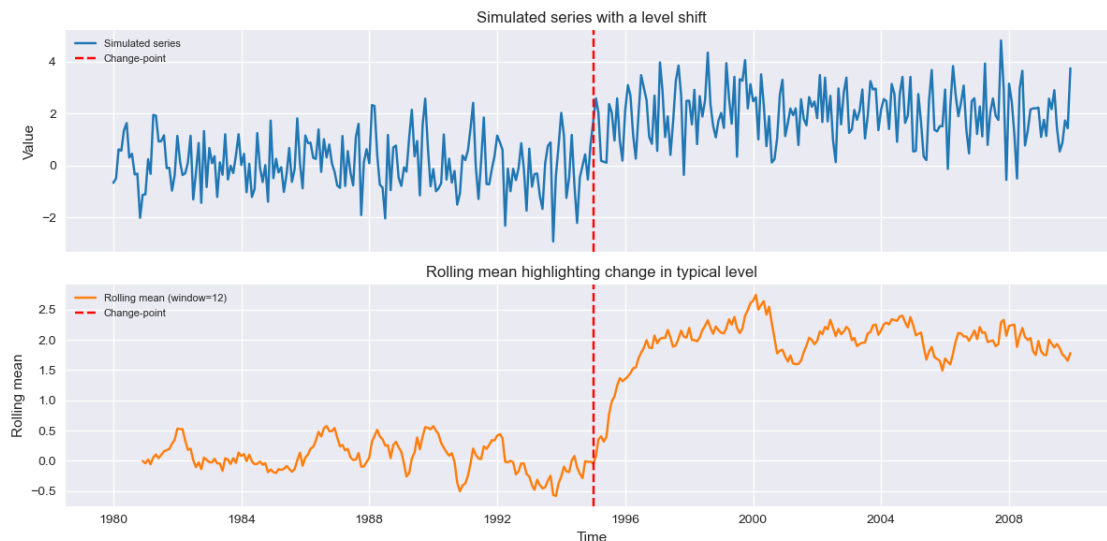
and illustrate:

- how the level change appears in a time series plot,
- how a rolling mean can help to visualise changes in typical level,
- how this differs from short-lived spikes or outliers.

Simulated level-shift example:

Total length: 360

Change-point at: 1995-01-01 00:00:00

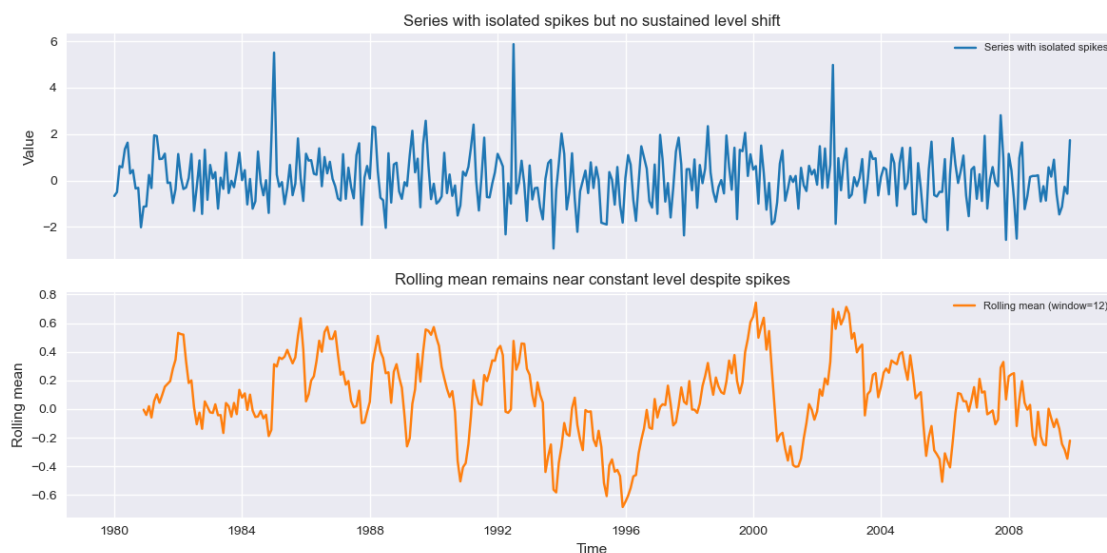


0.9.1 Rolling mean as a simple local level estimate

The 12-month rolling mean here is just the average over the most recent year:

$$\bar{X}_t^{(12)} = \frac{1}{12} \sum_{j=0}^{11} X_{t-j}.$$

It provides a **local estimate of typical level**, which makes the level shift easier to see. As in other sections, this is an exploratory tool: it helps visualise changes, but does not replace more formal change-point or regime-shift methods.



0.10 9. Mini-project: how to use these templates

For the mini-project, you can:

- start from one of the templates above (univariate, multivariate, or eqexp),
- adapt the `..._path` and column names to your own CSV file(s),
- repeat the basic steps:
 - structural checks and basic data quality,
 - plots of levels and differences,
 - rolling statistics,
 - ACF/PACF,
 - simple decomposition where appropriate,
 - persistence baseline (for forecasting-type problems),
 - visual comparison of series from different groups (for classification-type problems).

In your own data, you may see features that are not present in these curated examples, such as:

- irregular or changing sampling intervals,
- missing periods (gaps) or blocks of missing values,
- obvious level shifts or regime changes,
- sensor glitches or physically impossible values.

The intended workflow is:

1. Detect and document such features using simple diagnostic tools (missingness indicators, gap plots, rolling mean/variance, and similar).
2. Discuss serious features with someone who understands the domain and how the data were collected (for example, an experimentalist, clinician, or engineer).
3. Decide whether:
 - to restrict attention to a clean subset of the data,
 - to treat the detection of these features as an important part of your mini-project story,
 - or to make conservative modelling choices that explicitly acknowledge the limitations.

This companion focuses on inspecting and describing time series, not on developing full imputation schemes or specialised models for structural breaks and regime changes. When there is substantial missingness or clear changes in behaviour, sensible modelling choices depend on the context and on domain knowledge.

For the purposes of this mini-project, it is often preferable to:

- document clearly what kinds of features you see (gaps, level shifts, obvious glitches), and
- be explicit about how these limitations affect the questions you can and cannot answer,

rather than quietly applying ad-hoc transformations and then proceeding as if the series were well-behaved and stationary when it is not.

Change-point and regime-shift models, when they are used, are tools for describing and detecting structural changes. They do not turn an inherently unstable or evolving process into a simple stationary one.

Later companions will build on this foundation with more advanced models and, separately, spectral (frequency-domain) methods.